Programming Assignment: Tomasulo Algorithm with Multiple Issue

S M Masrur Ahmed
Assignment 1: Branch Predictor Implementation
ID: 2393372
Email: sahmed75@uh.edu

1 Introduction

Tomasulo Algorithm Tomasulo's algorithm lets ready instructions execute without waiting for earlier instructions that are still blocked. Register renaming handles WAR and WAW hazards, and the reorder buffer (ROB) keeps commit in program order.

I started with the supplied Python implementation, `tomasulo-python`, which issues one instruction per cycle. I kept its files, data structures and stage functions, and made the following changes:

- I added support for issuing 1 to 4 instructions per cycle in program order. I read the width from the input file (`Issue width = 2`) and use 1 by default. I also let commit retire up to the same number of instructions per cycle.
- I added the branch predictor described in the handout: a 1-bit predictor with an 8-entry BTB. I keep issuing on the predicted path, and on a misprediction I squash the younger instructions, rebuild the RAT and fetch the correct path two cycles later.
- I added a reader for the input format of the handout, so the configuration, initial registers and memory are no longer hard coded in `main.py`, and I print the registers and non-zero memory at the end.
- I fixed bugs in the starter code that broke results once several instructions were in flight, for example a broadcast that overwrote a newer register mapping.
- I tested 11 programs at every issue width and 1000 random programs, and compared every run with a sequential model.

What Works

- I can issue 1 to 4 instructions per cycle in order, including dependent instructions in the same issue group.
- I commit up to the issue width per cycle, in program order.
- I support `Beq` and `Bne` (offset in words, as in the ISA table) with a 1-bit predictor and an 8-entry BTB indexed by the low 3 bits of the word address. After a misprediction I squash the wrong-path instructions, recover the RAT, and fetch the correct path two cycles after the misprediction is found.
- I use one CDB, as in the handout. The integer adder is unpipelined and the FP units are pipelined. The number of reservation stations, latencies and FUs come from the input.
- I store single precision values in memory and print the timing table, all registers, the non-zero memory and the total number of cycles.

Limitations

- When I squash, I remove the wrong-path instructions from the functional units and cancel a wrong-path load in memory, so they do not keep those units busy.
- I update the BTB when a branch resolves, even if that branch is squashed later.
- I kept the starter's load/store queue rules: a load forwards only from the nearest older store, and it waits for memory until all older store addresses are known.

- I use a single address adder for loads and stores, so the `# of FUs` value of the load/store unit is not used.
- The CDB takes results in the order they finish, as in the starter code, so an older instruction can wait behind a younger one.

2 How I Changed the Starter Code

I kept the structure of the starter code: one function per stage, the same ROB, RAT, reservation station and load/store queue structures, and the same main loop. Most of my changes are small and local. The full set of changes is in `changes_from_starter.diff`, which adds 365 lines and removes 143 lines.

While adding multiple issue, I found that some problems in the starter code only showed up once several instructions were in flight at the same time. For example, `fu_exe` is called twice in a cycle when a new instruction starts, and the second call advanced every instruction in the unit again. I describe each fix next to the change that needed it.

3 Code Changes

3.1 Reading the input file and the issue width

The starter code read only the instructions from `code.in` and set everything else in `main.py`:

```
instructions = read_instruction('code.in')
'''initialize regs, RAT, memory'''
# integer REG
reg_int = []
for i in range(32):
    reg_int.extend([0])
reg_int[1] = 12
reg_int[2] = 32
...
size_rs_int_adder = 4
rs_int_adder = build_rs(size_rs_int_adder)
```

I added `read_input` to `init.py`. It reads the format from the handout, ignores case and allows commas between operands. It returns the instructions as `Op A B C`, the format the rest of the starter code already expects:

```
def read_input(path):
    config = {'Integer adder': [2, 1, 1], 'FP adder': [3, 3, 1], 'FP multiplier': [2, 20, 1],
              'Load/store unit': [3, 1, 4, 1], 'ROB entries': 128, 'Issue width': 1}
    ops = {'ld': 'Ld', 'sd': 'Sd', 'add': 'Add', 'addi': 'Addi', 'sub': 'Sub', 'add.d': 'Add.d',
           'sub.d': 'Sub.d', 'mult.d': 'Mult.d', 'bne': 'Bne', 'beq': 'Beq'}
    regs = {}
    memory = {}
    instructions = []
    with open(path) as f:
        lines = f.read().splitlines()
    for line in lines:
        line = line.strip()
        lower = line.lower()
        if (line == '') | line.startswith('#'):
            continue
        unit = [name for name in config if lower.startswith(name.lower())]
        if len(unit) > 0:
            numbers = [int(x) for x in line[len(unit[0]):].replace('=', ' ').split()]
            if unit[0] in ('ROB entries', 'Issue width'):
                config[unit[0]] = numbers[0]
            else:
                config[unit[0]][:len(numbers)] = numbers
        elif lower.startswith('mem['):
            for item in line.split(','):
                address, value = item.split('=')
                memory[int(address.strip()[4:-1])] = to_single(float(value))
        elif ('=' in line) & (lower[0] in 'rf'):
            for item in line.split(','):
                name, value = item.split('=')
                name = name.strip().upper()
                regs[name] = int(value) if name[0] == 'R' else float(value)
        else:
            tokens = line.replace(',', ' ').split()
            if tokens[0].lower() not in ops:
                raise ValueError('unknown instruction: ' + line)
            instructions.append(' '.join([ops[tokens[0].lower()]] + [t.upper() for t in tokens[1:]]))
    if not 1 <= config['Issue width'] <= 4:
        raise ValueError('issue width must be between 1 and 4')
    return config, regs, memory, instructions
```

In `main.py` I use these values instead of the hard coded ones. I also accept the issue width as a second command-line argument, so I can run the same input at different widths:

```
input_file = sys.argv[1] if len(sys.argv) > 1 else 'test_case.txt'
config, init_regs, init_memory, instructions = read_input(input_file)
issue_width = int(sys.argv[2]) if len(sys.argv) > 2 else config['Issue width']
if not 1 <= issue_width <= 4:
    sys.exit('issue width must be between 1 and 4')
...
size_rs_int_adder = config['Integer adder'][0]
rs_int_adder = build_rs(size_rs_int_adder)
```

3.2 Issuing multiple instructions

The starter `issue` function fetched exactly one instruction:

```
def issue(cycle, PC, instructions, ROB, size_ROB,
        rs_int_adder,
        rs_fp_adder,
        rs_fp_multi,
        ld_sd_queue, size_ld_sd_queue,
        rat_int, rat_fp):
    # fetch 1 instruction
    ins = instructions[PC.PC]
    op = ins.split(' ')[0]
    '''decode'''
    # LD/SD instructions
    if (op=='Ld')|(op=='Sd'):
    ...
```

I kept that function unchanged and renamed it to `issue_one_instruction`. The new `issue` calls it up to `issue_width` times. I stop when an instruction cannot get a ROB entry, reservation station or load/store queue entry, because the instructions after it have to wait too. I also end the group after a branch that is predicted taken, because the next instruction comes from the branch target. Each instruction updates the RAT before the next one is issued, so a dependent instruction in the same group reads the correct ROB tag:

```
def issue(cycle, PC, instructions, ROB, size_ROB,
        rs_int_adder,
        rs_fp_adder,
        rs_fp_multi,
        ld_sd_queue, size_ld_sd_queue,
        rat_int, rat_fp, issue_width):
    # after a misprediction, fetch starts again two cycles after it was found
    if cycle < PC.resume:
        return
    for i in range(issue_width):
        # stop at the end of the program
        if (PC.PC < 0) | (PC.PC >= len(instructions)) | (PC.valid == 0):
            break
        rob_size = len(ROB)
        pc = PC.PC
        issue_one_instruction(cycle, PC, instructions, ROB, size_ROB,
                rs_int_adder, rs_fp_adder, rs_fp_multi,
                ld_sd_queue, size_ld_sd_queue, rat_int, rat_fp)
        # stop when the instruction could not be issued (no ROB/RS/queue entry);
        # the instructions after it have to wait too
        if len(ROB) == rob_size:
            break
        # a branch predicted taken ends the group; fetch continues at its target
        if PC.PC != pc + 1:
            break
```

3.3 Committing multiple instructions

The starter `commit` handled one instruction, with special cases for a branch and for a store followed by another instruction. It also wrote the commit cycle as `cycle+1` while removing the instruction in the current cycle:

```
def commit(ROB, reg_int, reg_fp, cycle, instructions):
    if len(ROB)>0:
        if instructions[ROB[0].PC].split(' ')[0]=='Bne':
            if len(ROB[0].exe)!=0:
                entry = ROB.popleft()
                print_ROB(entry, instructions)
        if (len(ROB)>0)&(instructions[ROB[0].PC].split(' ')[0]!='Bne'):
            if len(ROB[0].cdb)!=0: # broadcasted instructions
                ROB[0].commit.append(cycle+1)
```

```
                entry = ROB.popleft()
                modify_arch_reg(entry, reg_int, reg_fp)
                print_ROB(entry, instructions)
            elif len(ROB[0].commit)!=0: # Sd
                entry = ROB.popleft()
                print_ROB(entry, instructions)
                if len(ROB)>0:
                    if len(ROB[0].cdb)!=0: # broadcasted instructions
                        ROB[0].commit.append(cycle+1)
                        entry = ROB.popleft()
                        modify_arch_reg(entry, reg_int, reg_fp)
                        print_ROB(entry, instructions)
            else:
                pass
```

I commit up to `commit_width` instructions (the issue width) from the ROB head in program order. An instruction commits in a cycle after its write back, and a branch in a cycle after its EX stage has finished. The starter committed a branch as soon as its EX had started. I now record the cycle in which the instruction actually leaves the ROB, so the COMMIT column never shows more commits in a cycle than the commit width:

```
def commit(ROB, reg_int, reg_fp, cycle, instructions, commit_width, committed):
    for i in range(commit_width):
        if len(ROB)==0:
            break
        op = instructions[ROB[0].PC].split(' ')[0]
        if (op=='Bne')|(op=='Beq'):
            # a branch commits in a cycle after its EX has finished
            if (len(ROB[0].exe)!=2) or (ROB[0].exe[1] >= cycle):
                break
            ROB[0].commit.append(cycle)
            entry = ROB.popleft()
        elif (len(ROB[0].cdb)!=0) and (ROB[0].cdb[0] < cycle): # broadcasted in an earlier cycle
            ROB[0].commit.append(cycle)
            entry = ROB.popleft()
            modify_arch_reg(entry, reg_int, reg_fp)
        elif len(ROB[0].commit)!=0: # Sd
            entry = ROB.popleft()
        else:
            break
        print_ROB(entry, instructions)
        committed.append(entry)
```

3.4 Execution units

The starter picked the first ready instruction in a reservation station list and sent one instruction per unit and cycle to the functional unit. It also called `fu_exe` a second time for that instruction, which advanced every other instruction in the unit twice:

```
def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
    if len(fu)!=0:
        for element in fu:
            element.cycle += 1
        # write starting cycle
        if fu[-1].cycle == 1:
            index = find_ROB_entry(ROB, fu[-1].dest_tag)
            ROB[index].exe.append(cycle)
        # finish cycle
        if fu[0].cycle == time_fu:
            index = find_ROB_entry(ROB, fu[0].dest_tag)
            ROB[index].exe.append(cycle)
    ...
```

```
# from rs
# int_adder
# fetch valid instruction
if check_valid_ins_in_rs(rs_int_adder)>=0:
    index = check_valid_ins_in_rs(rs_int_adder)
    fu_int_adder.append(fu_entry())
    add_entry_into_fu(fu_int_adder, rs_int_adder[index])
    # check if it's a waiting instruction
    if ROB[find_ROB_entry(ROB, rs_int_adder[index].dest_tag)].issue[0] < cycle:
        fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
    # remove ins from rs
    rs_int_adder[index].busy = 0
...
```

In my version I advance every entry only once per cycle, and I record the start cycle for each entry that starts, not only for the last one. I also let more than one entry finish in a cycle, which happens when a unit has more than one FU:

```
def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
    if len(fu)!=0:
        for element in fu:
            # fu_exe is called twice in a cycle when new instructions start,
            # so every entry is advanced only once per cycle
            if getattr(element, 'last_cycle', 0) != cycle:
                element.cycle += 1
                element.last_cycle = cycle
                # write starting cycle
                if element.cycle == 1:
                    index = find_ROB_entry(ROB, element.dest_tag)
                    ROB[index].exe.append(cycle)
        # finish cycle (several entries can finish together with more than one FU)
        while (len(fu)!=0) and (fu[0].cycle == time_fu):
            index = find_ROB_entry(ROB, fu[0].dest_tag)
            ROB[index].exe.append(cycle)
    ...
```

I pick the oldest ready instruction that was issued in an earlier cycle:

```
def check_valid_ins_in_rs(rs, ROB, cycle):
    index = -1
    oldest = len(ROB)
    for i in range(len(rs)):
        if (rs[i].valid_1st==1)&(rs[i].valid_2nd==1)&(rs[i].busy==1):
            position = find_ROB_entry(ROB, rs[i].dest_tag)
            if (ROB[position].issue[0] < cycle) & (position < oldest):
                index = i
                oldest = position
    return index
```

I kept the starter's three fetch blocks, one per unit, and put each one in a loop so a unit can start up to `# of FUs` instructions per cycle. The integer adder is unpipelined, so its loop only uses the FUs that were free at the start of the cycle. The FP adder and multiplier blocks are the same without that check:

```
# from rs (up to one instruction per FU and cycle)
# int_adder: unpipelined, so only FUs that were free at the start of the cycle
# fetch valid instruction
for i in range(num_fu[0] - int_adder_busy):
    if check_valid_ins_in_rs(rs_int_adder, ROB, cycle)>=0:
        index = check_valid_ins_in_rs(rs_int_adder, ROB, cycle)
        fu_int_adder.append(fu_entry())
        add_entry_into_fu(fu_int_adder, rs_int_adder[index])
        # check if it's a waiting instruction
        if ROB[find_ROB_entry(ROB, rs_int_adder[index].dest_tag)].issue[0] < cycle:
            fu_exe(fu_int_adder, results_buffer, ROB, time_fu_int_adder, cycle, PC)
        # remove ins from rs
        rs_int_adder[index].busy = 0
...
```

The address adder for loads and stores had a similar problem. It only counted the first cycle of an address calculation, so a latency above 1 never finished, and it could start a new address in the same cycle the previous one finished. I count every cycle and only start a new address when the adder was free at the start of the cycle:

```
def ld_sd_execution(ld_sd_exe, time_ld_sd_exe, ld_sd_queue, ROB, cycle):
    if ld_sd_exe.busy == 1:
        # write down starting cycle
        if ld_sd_exe.cycle == 0:
            for element in ld_sd_queue:
                if element.ld_sd_tag == ld_sd_exe.dest_tag:
                    break
            index = find_ROB_entry(ROB, element.dest_tag)
            ROB[index].exe.append(cycle)
        # execute (every cycle, so address calculation can take more than 1 cycle)
        ld_sd_exe.cycle += 1
    ...

...
# units that are busy at the start of the cycle cannot start a new
# instruction in the same cycle (address adder and unpipelined integer adder)
ld_sd_busy = ld_sd_exe.busy
int_adder_busy = len(fu_int_adder)
...
if (index>=0)&(ld_sd_exe.busy==0)&(ld_sd_busy==0):
```

3.5 Write back and the RAT

The starter updated the RAT with the broadcast value every time:

```
# rat_int, rat_fp, reservation stations, ld_sd_queue, ROB
def boroadcast(cdb, rat_int, rat_fp,
               rs_int_adder, rs_fp_adder, rs_fp_multi,
               ld_sd_queue, ROB, cycle):
    # dest_tag
    dest_tag = cdb.dest_tag
    index = find_ROB_entry(ROB, dest_tag)
    reg_tag = ROB[index].dest_tag
    # rat_int
    if reg_tag[0] == 'R':
        rat_int[int(reg_tag[1:])] = cdb.value
    # rat_fp
    if reg_tag[0] == 'F':
        rat_fp[int(reg_tag[1:])] = cdb.value
```

This is wrong when a younger instruction has renamed the same register in the meantime (a WAW hazard). The broadcast then overwrote the younger mapping, and later instructions read the old value. With more instructions in flight this happened in my tests, so I only update the RAT when it still points to the instruction that is broadcasting:

```
# rat_int / rat_fp: only if no younger instruction has renamed the register since
if reg_tag[0] == 'R':
    if rat_int[int(reg_tag[1:])] == dest_tag:
        rat_int[int(reg_tag[1:])] = cdb.value
# rat_fp
if reg_tag[0] == 'F':
    if rat_fp[int(reg_tag[1:])] == dest_tag:
        rat_fp[int(reg_tag[1:])] = cdb.value
```

3.6 Load/store queue

In `mem.py` the function `check_if_sd_committable` was used without being called, so the condition was always true, and it read a field name that does not exist (`ldsd_tag`). The loop that chooses a load for memory also checked the data of the first queue entry instead of the load it was looking at:

```
def check_if_sd_committable(ld_sd_queue, ROB):
    flag = False
    if ld_sd_queue[0].ldsd_tag == ROB[0].dest_tag:
        flag = True
    if ld_sd_queue[0].ldsd_tag == ROB[1].dest_tag:
        # previous ins has been broadcasted
        if len(ROB[0].cdb)!=0:
            flag = True
    return flag

...
if (check_if_sd_committable):
...
if (type(ld_sd_queue[0].data)!=int)&(type(ld_sd_queue[0].data)!=float):
```

I fixed the call and the field name, and I only let a store write memory when it is at the head of the ROB, which is when it commits:

```
def check_if_sd_committable(ld_sd_queue, ROB):
    flag = False
    # a store commits (and writes memory) only from the head of the ROB, so the
    # COMMIT column stays in program order and within the commit width
    if ld_sd_queue[0].ld_sd_tag == ROB[0].dest_tag:
        flag = True
    return flag

...
if (check_if_sd_committable(ld_sd_queue, ROB)):
...
if (type(ld_sd_queue[index].data)!=int)&(type(ld_sd_queue[index].data)!=float):
```

A forwarded load gets its value in the same cycle it is checked, so I only forward with addresses that were calculated in an earlier cycle:

```
def address_ready(entry, cycle):
    return (entry.ready == 1) and (entry.ready_cycle < cycle)
```

3.7 Branch prediction and recovery

The starter code did not predict branches. When it issued a `Bne`, it set `PC.valid` to 0 and issued nothing until the branch was resolved:

```
# Bne
elif (op=='Bne'):
    # check space
    if (len(ROB)<size_ROB)&(check_rs_space(rs_int_adder)>=0):
        # put ins into ROB
        put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
        # put ins into rs_int_adder
        index = check_rs_space(rs_int_adder)
        put_bne_into_rs(rs_int_adder, index, ins, rat_int, rat_fp, ROB)
        # PC
        PC.valid = 0
```

I added the predictor from the handout. The BTB has 8 entries and is indexed with the low 3 bits of the word address of the branch. Each entry holds the branch address, the target and a 1-bit prediction. On a BTB miss I predict not taken. I keep the BTB in the `PC` object, because it belongs to the fetch unit and `PC` is already passed to the issue and EX stages:

```
def predict_branch(PC, pc):
    entry = PC.btb[pc % 8]
    if (entry is not None) and (entry[0] == pc) and (entry[2] == 1):
        return entry[1]
    return pc + 1
```

At issue I store the predicted next PC in the ROB entry and continue on that path:

```
# Bne/Beq
elif (op=='Bne')|(op=='Beq'):
    # check space
    if (len(ROB)<size_ROB)&(check_rs_space(rs_int_adder)>=0):
        # put ins into ROB
        put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
        # put ins into rs_int_adder
        index = check_rs_space(rs_int_adder)
        put_bne_into_rs(rs_int_adder, index, ins, rat_int, rat_fp, ROB)
        # PC: continue on the predicted path
        ROB[-1].pred_next = predict_branch(PC, PC.PC)
        PC.PC = ROB[-1].pred_next
```

The starter resolved a branch at the end of EX by setting the PC. It only supported `Bne` and treated the offset as bytes:

```
elif (fu[0].op=='Bne'):
    fu_results.pop()
    if fu[0].value1 == fu[0].value2:
        PC.PC += 1
        PC.valid = 1
    else:
        index = find_ROB_entry(ROB, fu[0].dest_tag)
        offset = ROB[index].dest_tag
        PC.PC = int(PC.PC + 1 + offset/4)
```

In my version I add `Beq`, use the offset in words (`PC+4+offset<<2`), update the BTB, and compare the real next PC with the prediction. If they differ, I mark the branch. When two branches are mispredicted in the same cycle, I keep the older one:

```
elif (fu[0].op=='Bne')|(fu[0].op=='Beq'):
    fu_results.pop()
    index = find_ROB_entry(ROB, fu[0].dest_tag)
    branch_pc = ROB[index].PC
    # target = PC + 4 + offset<<2, so the offset is in words
    target = branch_pc + 1 + ROB[index].dest_tag
    taken = (fu[0].value1 == fu[0].value2) == (fu[0].op=='Beq')
    # update the BTB entry and its 1-bit predictor
    PC.btb[branch_pc % 8] = [branch_pc, target, 1 if taken else 0]
    next_pc = target if taken else branch_pc + 1
    # misprediction: keep the oldest one found in this cycle;
    # main.py calls recover() after the EX stage
    if next_pc != ROB[index].pred_next:
        ROB[index].mispredicted = True
        if (PC.mispredict == '') or (index < find_ROB_entry(ROB, PC.mispredict)):
            PC.mispredict = ROB[index].ROB_tag
            PC.redirect = next_pc
```

Right after the EX stage, `main.py` calls `recover` when a branch was mispredicted:

```
# a branch was mispredicted at the end of EX: squash the younger instructions
if PC.mispredict != '':
    recover(PC, ROB, rat_int, rat_fp, reg_int, reg_fp,
            rs_int_adder, rs_fp_adder, rs_fp_multi,
            fu_int_adder, fu_fp_adder, fu_fp_multi,
            ld_sd_queue, ld_sd_exe, ld_sd_mem, results_buffer, cdb, cycle)
```

`recover` removes every instruction after the branch from the ROB, the reservation stations, the functional units, the finished results, the CDB and the load/store queue. Then it rebuilds the RAT from the committed registers and the instructions left in the ROB. It does not cancel a store in memory: a store only writes memory after it commits, so it is never on a wrong path, but its old ROB tag can already be in use again. Fetch starts again in cycle $n+2$ when the misprediction is found in cycle n, as in the handout:

```
def recover(PC, ROB, rat_int, rat_fp, reg_int, reg_fp,
            rs_int_adder, rs_fp_adder, rs_fp_multi,
            fu_int_adder, fu_fp_adder, fu_fp_multi,
            ld_sd_queue, ld_sd_exe, ld_sd_mem, results_buffer, cdb, cycle):
    # ROB: remove the instructions after the branch
    branch = find_ROB_entry(ROB, PC.mispredict)
    squashed = []
    while len(ROB) > branch + 1:
        squashed.append(ROB.pop().ROB_tag)
    # reservation stations, functional units, finished results and the CDB
    for rs in (rs_int_adder, rs_fp_adder, rs_fp_multi):
        for entry in rs:
            if (entry.busy == 1) and (entry.dest_tag in squashed):
                entry.busy = 0
    for fu in (fu_int_adder, fu_fp_adder, fu_fp_multi):
        for entry in list(fu):
            if entry.dest_tag in squashed:
                fu.remove(entry)
    for entry in list(results_buffer):
        if entry.dest_tag in squashed:
            results_buffer.remove(entry)
    if (cdb.valid == 1) and (cdb.dest_tag in squashed):
        cdb.valid = 0
    # load/store queue, address adder and memory
    for entry in list(ld_sd_queue):
        if entry.dest_tag in squashed:
            ld_sd_queue.remove(entry)
    if (ld_sd_exe.busy == 1) and (ld_sd_exe.dest_tag not in [e.ld_sd_tag for e in ld_sd_queue]):
        ld_sd_exe.busy = 0
    # (a store in memory has already committed; its old ROB tag may be reused)
    if (ld_sd_mem.busy == 1) and (ld_sd_mem.op == 'Ld') and (ld_sd_mem.dest_tag in squashed):
        ld_sd_mem.busy = 0
    # RAT: committed registers, then the instructions still in the ROB
    for i in range(32):
        rat_int[i] = reg_int[i]
        rat_fp[i] = reg_fp[i]
    for entry in ROB:
        dest = entry.dest_tag
        if (type(dest) == str) and (dest[0] in 'RF') and (dest != 'R0'):
            rat = rat_int if dest[0] == 'R' else rat_fp
            rat[int(dest[1:])] = entry.value if len(entry.cdb) != 0 else entry.ROB_tag
    # fetch the correct path in cycle + 2
    PC.PC = PC.redirect
    PC.resume = cycle + 2
    PC.mispredict = ''
```

Because the offset is now in words, I changed the offset in the starter's `test_case.txt` from -12 to -3, which jumps to the same instruction as before.

3.8 Main loop and output

The starter stopped as soon as the ROB was empty, even if there were more instructions to issue (for example right after a branch), and it could stop before the last store had written memory:

```
# main
while (len(ROB)>0)|(cycle==1):

    '''ISSUE stage'''
    if (PC.PC<len(instructions))&(PC.valid==1):
        issue(cycle, PC, instructions, ROB, size_ROB,
                rs_int_adder,
                rs_fp_adder,
                rs_fp_multi,
                ld_sd_queue, size_ld_sd_queue,
                rat_int, rat_fp)
```

I keep running while there is something in the ROB, something left to issue, or a memory write in progress. At the end I print the registers, the non-zero memory and the total cycles:

```
# main: run until the ROB is empty, there is nothing left to issue,
# and the last store has finished writing memory
while (len(ROB)>0)|((PC.PC>=0)&(PC.PC<len(instructions)))|(ld_sd_mem.busy==1):
    if cycle > 100000:
```

```
        sys.exit('program did not finish within 100000 cycles')

    '''ISSUE stage'''
    if (PC.PC<len(instructions))&(PC.valid==1):
        issue(cycle, PC, instructions, ROB, size_ROB,
                rs_int_adder,
                rs_fp_adder,
                rs_fp_multi,
                ld_sd_queue, size_ld_sd_queue,
                rat_int, rat_fp, issue_width)
```

```
# results
print('')
print_registers(reg_int, reg_fp)
print('')
print_memory(memory)
print('')
# total cycles: the last commit or the end of the last memory write, whichever is later
total_cycles = cycle - 1
for entry in committed:
    total_cycles = max([total_cycles] + entry.commit + entry.mem)
print('Issue width: %d   Total cycles: %d   Instructions committed: %d'
      % (issue_width, total_cycles, len(committed)))
```

I also made a few smaller fixes: `R0` is never renamed or written, an address like `8(R10)` works (the starter only handled one-digit registers), and stores round their value to single precision.

4 Testing

I checked the simulator against `tests/golden.py`, a sequential model that executes the same program without a pipeline. In `tests/run_tests.py`, I run every test with `main.py` at issue widths 1, 2, 3 and 4 and check its printed output:

- I get the same final registers, memory and committed instruction sequence as the sequential model.
- EX starts after issue and lasts for the configured latency, and an instruction only uses a register after the cycle in which it was broadcast.
- Issue and commit stay in order and within the issue width per cycle, and there is at most one broadcast per cycle.
- The unpipelined integer adder, the address adder and the memory port are never used by two instructions in the same cycle.
- After a mispredicted branch, the next instruction is issued at least 2 cycles after the branch was resolved.
- The total cycle count is the later of the last commit and the last memory access.

I also generated random loops with ALU instructions, loads, stores to a few addresses, and forward branches, and ran each one on a random configuration at all four widths:

```
random programs: 1000 programs x 4 widths, 0 failures (7304 branch mispredictions recovered)
ALL TESTS PASSED
```

To check that these tests catch mistakes, I broke the simulator in nine ways, one at a time, and ran the tests with 300 random programs. Every one was caught:

- undoing the RAT fix in write back: 75 failures;
- advancing a functional unit twice in a cycle: 422 failures;
- forwarding with an address from the same cycle: 58 failures;
- issuing past an instruction that could not be issued: 1155 failures;
- committing 4 instructions per cycle regardless of the width: 635 failures;
- not rebuilding the RAT after a misprediction: 595 failures;
- leaving squashed instructions in the functional units: 201 failures;
- fetching the correct path one cycle early: 942 failures;
- cancelling a committed store during recovery: 33 failures.

5 Results

I show the results at issue widths 1 to 4 below. Since I got the same final registers and memory at every width, I list those values once for each test case.

5.1 Summary of all test cases

I list the total cycles for all 11 programs in Table 1.

Going from width 1 to 2 helped in most tests, and wider issue helped less. Three things limit the gain. First, there is one CDB, so at most one result is written back per cycle. In `ilp_independent`, the 16 independent instructions take 22 cycles at width 1 and 19 at width 4. Second, the 1-bit predictor mispredicts the first and last time a loop branch runs and whenever an if/else changes direction, and every misprediction costs the instructions fetched on the wrong path. In `branch_loop`, the time goes from 52 to 41 cycles, with 5 mispredictions at width 4. Third, long dependence chains do not get faster with wider issue: `loop_daxpy` takes 56 cycles at every width. Each iteration has a chain of a load, a 15-cycle multiply, an add and a store, and all loads and stores share the single memory port, so issuing faster does not shorten the slowest path.

I also ran `sample_report_tc1` and `sample_report_tc2` from the sample report. I get 36, 35, 35 and 35 cycles for test case 1, compared to 36, 33, 33 and 33 in the sample report, and 49, 47, 47 and 47 for test case 2, compared to 47, 46, 46 and 46. The sample report writes back as many results per cycle as the issue width, while I use the single CDB from the handout, so I expected my numbers to be higher at widths above 1. Test case 2 is also slower at width 1. I did not find the exact reason, but the sample report's tables show memory accesses that overlap (for example cycles 3–7 and 5–9 at width 2), which the single-ported, unpipelined memory in the handout does not allow.

5.2 Test Case #01: Example from the handout

I used the handout's configuration and program, but I added an `Addi` counter, because the original loop never ends (`R2` is never equal to `R3`).

Configuration of the Architecture

```
               # of rs    Cycles in EX    Cycles in Mem    # of FUs
Integer adder    2          1                               1
FP adder         3          3                               1
FP multiplier    2          20                              1
Load/store unit  3          1               4               1

ROB entries = 128
Issue width = 2
R1=10, R2=20, F2=30.1
Mem[4]=1, Mem[8]=2, Mem[12]=3.4
```

List of Instructions

```
Add.d F1, F2, F3
Ld F4, 8(R1)
Addi R2, R2, -10
Bne R2, R3, -4
```

Final values	Integer registers (non-zero)	R1 = 10
	FP registers (non-zero)	F1 = 30.1, F2 = 30.1
	Memory (non-zero)	Mem[4] = 1, Mem[8] = 2, Mem[12] = 3.4

The first `Bne` is not in the BTB yet, so I predict it not taken. At width 1 it resolves taken at the end of cycle 7, and I issue the correct path in cycle 9, two cycles later. At width 2 it resolves in cycle 5 and the correct path is issued in cycle 7. The BTB then predicts the second `Bne` taken, but it falls through, so I squash the instructions fetched from the top of the loop. They do not appear in the tables, which only list committed instructions. The load uses address $8 + 10 = 18$, which holds 0, so `F4` ends up as 0. The program takes 19, 15, 13 and 13 cycles at widths 1 to 4.

Test	Width 1	Width 2	Width 3	Width 4
branch_loop	52	41	41	41
forwarding_disambiguation	20	18	18	16
handout_sample	19	15	13	13
hazards_r0	22	18	18	18
ilp_independent	22	19	19	19
loop_daxpy	56	56	56	56
sample_report_tc1	36	35	35	35
sample_report_tc2	49	47	47	47
single_precision_store_drain	20	19	19	19
starter_test_case	43	37	36	36
structural_hazards	33	30	30	30

Table 1: Total cycles for every test program.

	ISSUE	EX	MEM	WB	COMMIT
Add.d F1 F2 F3	1	2–4		5	6
Ld F4 8(R1)	2	3–3	4–7	8	9
Addi R2 R2 -10	3	4–4		6	10
Bne R2 R3 -4	4	7–7			11
Add.d F1 F2 F3	9	10–12		13	14
Ld F4 8(R1)	10	11–11	12–15	16	17
Addi R2 R2 -10	11	12–12		14	18
Bne R2 R3 -4	12	15–15			19

Figure 1: Simulation results for Test Case #01 with issue width 1 (19 cycles).

	ISSUE	EX	MEM	WB	COMMIT
Add.d F1 F2 F3	1	2–4		5	6
Ld F4 8(R1)	1	2–2	3–6	7	8
Addi R2 R2 -10	2	3–3		4	8
Bne R2 R3 -4	2	5–5			9
Add.d F1 F2 F3	7	8–10		11	12
Ld F4 8(R1)	7	8–8	9–12	13	14
Addi R2 R2 -10	8	9–9		10	14
Bne R2 R3 -4	8	11–11			15

Figure 2: Simulation results for Test Case #01 with issue width 2 (15 cycles).

	ISSUE	EX	MEM	WB	COMMIT
Add.d F1 F2 F3	1	2–4		5	6
Ld F4 8(R1)	1	2–2	3–6	7	8
Addi R2 R2 -10	1	2–2		3	8
Bne R2 R3 -4	2	4–4			8
Add.d F1 F2 F3	6	7–9		10	11
Ld F4 8(R1)	6	7–7	8–11	12	13
Addi R2 R2 -10	6	7–7		8	13
Bne R2 R3 -4	7	9–9			13

Figure 3: Simulation results for Test Case #01 with issue width 3 (13 cycles).

	ISSUE	EX	MEM	WB	COMMIT
Add.d F1 F2 F3	1	2–4		5	6
Ld F4 8(R1)	1	2–2	3–6	7	8
Addi R2 R2 -10	1	2–2		3	8
Bne R2 R3 -4	1	4–4			8
Add.d F1 F2 F3	6	7–9		10	11
Ld F4 8(R1)	6	7–7	8–11	12	13
Addi R2 R2 -10	6	7–7		8	13
Bne R2 R3 -4	6	9–9			13

Figure 4: Simulation results for Test Case #01 with issue width 4 (13 cycles).

5.3 Test Case #02: The test case from the starter code

This is `test_case.txt` from the starter code, with the branch offset written in words.

Configuration of the Architecture

```
               # of rs         Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4               1                               1
FP adder        3               4                               1
FP multiplier   2               15                              1
Load/store unit 5               1               5               1

ROB entries = 64
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0
```

List of Instructions

```
Ld F2 0(R1)
Mult.d F4 F2 F20
Ld F6 0(R2)
Add.d F6 F4 F6
Sd F6 0(R2)
Addi R1 R1 -4
Addi R2 R2 -4
Bne R1 R0 -3
Add.d F20 F2 F2
```

Final values

Integer registers (non-zero)	R2 = 20
FP registers (non-zero)	F2 = 1, F4 = 3, F6 = 7, F20 = 2
Memory (non-zero)	Mem[4] = 3, Mem[8] = 2, Mem[12] = 1, Mem[24] = 6, Mem[28] = 5, Mem[32] = 7

The branch jumps back to the two `Addi` instructions, so the loop runs three times and `R1` goes from 12 to 0. Wider issue lets me issue the first eight instructions in two cycles at width 4, but the `Mult.d` still has to wait for the load and takes 15 cycles, and the store waits for the add. The branch is mispredicted twice: the first time, because it is not in the BTB yet, and at the end of the loop, when it falls through. The time goes from 43 cycles at width 1 to 36 at width 4.

5.4 Test Case #03: Independent instructions

I wrote this test with 16 independent instructions spread across the units, so I could measure the benefit of wider issue without dependency stalls.

Configuration of the Architecture

```
               # of rs    Cycles in EX    Cycles in Mem    # of FUs
Integer adder    4          1                               2
FP adder         4          2                               2
FP multiplier    4          4                               2
Load/store unit  6          1               2               1

ROB entries = 64
R1=1, R2=2, F1=1.5, F2=2.5
Mem[0]=1, Mem[4]=2, Mem[8]=3, Mem[12]=4
```

	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	9	10
`Mult.d F4 F2 F20`	2	10–24		25	26
`Ld F6 0(R2)`	3	4–4	8–12	14	27
`Add.d F6 F4 F6`	4	26–29		30	31
`Sd F6 0(R2)`	5	6–6	33–37		33
`Addi R1 R1 -4`	6	7–7		8	34
`Addi R2 R2 -4`	7	8–8		10	35
`Bne R1 R0 -3`	8	9–9			36
`Addi R1 R1 -4`	11	12–12		13	37
`Addi R2 R2 -4`	12	13–13		15	38
`Bne R1 R0 -3`	13	14–14			39
`Addi R1 R1 -4`	14	15–15		16	40
`Addi R2 R2 -4`	15	16–16		17	41
`Bne R1 R0 -3`	16	17–17			42
`Add.d F20 F2 F2`	19	20–23		24	43

Figure 5: Simulation results for Test Case #02 with issue width 1 (43 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4 F2 F20`	1	9–23		24	25
`Ld F6 0(R2)`	2	3–3	8–12	14	25
`Add.d F6 F4 F6`	2	25–28		29	30
`Sd F6 0(R2)`	3	4–4	32–36		32
`Addi R1 R1 -4`	3	4–4		5	32
`Addi R2 R2 -4`	4	5–5		6	33
`Bne R1 R0 -3`	4	6–6			33
`Addi R1 R1 -4`	8	9–9		10	34
`Addi R2 R2 -4`	8	10–10		11	34
`Bne R1 R0 -3`	9	11–11			35
`Addi R1 R1 -4`	10	12–12		13	35
`Addi R2 R2 -4`	10	13–13		15	36
`Bne R1 R0 -3`	11	14–14			36
`Add.d F20 F2 F2`	16	17–20		21	37

Figure 6: Simulation results for Test Case #02 with issue width 2 (37 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4 F2 F20`	1	9–23		24	25
`Ld F6 0(R2)`	1	3–3	8–12	14	25
`Add.d F6 F4 F6`	2	25–28		29	30
`Sd F6 0(R2)`	2	4–4	32–36		32
`Addi R1 R1 -4`	2	3–3		4	32
`Addi R2 R2 -4`	3	4–4		5	32
`Bne R1 R0 -3`	3	5–5			33
`Addi R1 R1 -4`	7	8–8		9	33
`Addi R2 R2 -4`	7	9–9		10	33
`Bne R1 R0 -3`	7	10–10			34
`Addi R1 R1 -4`	8	11–11		12	34
`Addi R2 R2 -4`	9	12–12		13	34
`Bne R1 R0 -3`	10	13–13			35
`Add.d F20 F2 F2`	15	16–19		20	35

Figure 7: Simulation results for Test Case #02 with issue width 3 (36 cycles).

List of Instructions

```
Add R3, R1, R2
Add.d F3, F1, F2
Mult.d F4, F1, F2
Ld F5, 0(R0)
Sub R4, R2, R1
Sub.d F6, F2, F1
Mult.d F7, F2, F2
Ld F8, 4(R0)
Addi R5, R1, 10
Add.d F9, F1, F1
Mult.d F10, F1, F1
Ld F11, 8(R0)
Addi R6, R2, 20
Add.d F12, F2, F2
Mult.d F13, F2, F1
Ld F14, 12(R0)
```

Final values

Integer registers (non-zero)	R1 = 1, R2 = 2, R3 = 3, R4 = 1, R5 = 11, R6 = 22
FP registers (non-zero)	F1 = 1.5, F2 = 2.5, F3 = 4, F4 = 3.75, F5 = 1, F6 = 1, F7 = 6.25, F8 = 2, F9 = 3, F10 = 2.25, F11 = 3, F12 = 5, F13 = 3.75, F14 = 4
Memory (non-zero)	Mem[0] = 1, Mem[4] = 2, Mem[8] = 3, Mem[12] = 4

At width 4, I issue all 16 instructions in the first 4 cycles and they finish executing early, but each result needs its own cycle on the single CDB. There is exactly one write back in every cycle from 3 to 18, so the program takes 19 cycles, the same as at widths 2 and 3.

5.5 Test Case #04: Store-to-load forwarding

I used this test to check the load/store queue. The first store gets its data from a 6-cycle multiply, so the load after it waits for the data and then forwards it. The second store gets its address from a chain of four `Addi` instructions, so the load after it has to wait until the address is known. The first store and load use address 8, the second pair uses address 16, and in both pairs the load gets its value from the store.

Configuration of the Architecture

```
               # of rs    Cycles in EX    Cycles in Mem    # of FUs
Integer adder    2          1                               1
FP adder         2          2                               1
FP multiplier    2          6                               1
Load/store unit  6          1               3               1

ROB entries = 32
R1=8, F1=2.0, F2=5.0
Mem[8]=1.5, Mem[16]=7.0
```

List of Instructions

```
Mult.d F3, F1, F2
Sd F3, 0(R1)
Ld F4, 0(R1)
Addi R2, R0, 4
Addi R2, R2, 4
Addi R2, R2, 4
Addi R2, R2, 4
Sd F1, 0(R2)
Ld F5, 16(R0)
Add.d F6, F5, F4
```

	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4 F2 F20`	1	9–23		24	25
`Ld F6 0(R2)`	1	3–3	8–12	14	25
`Add.d F6 F4 F6`	1	25–28		29	30
`Sd F6 0(R2)`	2	4–4	32–36		32
`Addi R1 R1 -4`	2	3–3		4	32
`Addi R2 R2 -4`	2	4–4		5	32
`Bne R1 R0 -3`	2	5–5			32
`Addi R1 R1 -4`	7	8–8		9	33
`Addi R2 R2 -4`	7	9–9		10	33
`Bne R1 R0 -3`	7	10–10			33
`Addi R1 R1 -4`	8	11–11		12	33
`Addi R2 R2 -4`	9	12–12		13	34
`Bne R1 R0 -3`	10	13–13			34
`Add.d F20 F2 F2`	15	16–19		20	34

Figure 8: Simulation results for Test Case #02 with issue width 4 (36 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Add R3 R1 R2`	1	2–2		3	4
`Add.d F3 F1 F2`	2	3–4		5	6
`Mult.d F4 F1 F2`	3	4–7		8	9
`Ld F5 0(R0)`	4	5–5	6–7	9	10
`Sub R4 R2 R1`	5	6–6		7	11
`Sub.d F6 F2 F1`	6	7–8		10	12
`Mult.d F7 F2 F2`	7	8–11		12	13
`Ld F8 4(R0)`	8	9–9	10–11	13	14
`Addi R5 R1 10`	9	10–10		11	15
`Add.d F9 F1 F1`	10	11–12		14	16
`Mult.d F10 F1 F1`	11	12–15		16	17
`Ld F11 8(R0)`	12	13–13	14–15	17	18
`Addi R6 R2 20`	13	14–14		15	19
`Add.d F12 F2 F2`	14	15–16		18	20
`Mult.d F13 F2 F1`	15	16–19		20	21
`Ld F14 12(R0)`	16	17–17	18–19	21	22

Figure 9: Simulation results for Test Case #03 with issue width 1 (22 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Add R3 R1 R2`	1	2–2		3	4
`Add.d F3 F1 F2`	1	2–3		4	5
`Mult.d F4 F1 F2`	2	3–6		8	9
`Ld F5 0(R0)`	2	3–3	4–5	7	9
`Sub R4 R2 R1`	3	4–4		5	10
`Sub.d F6 F2 F1`	3	4–5		6	10
`Mult.d F7 F2 F2`	4	5–8		12	13
`Ld F8 4(R0)`	4	5–5	6–7	11	13
`Addi R5 R1 10`	5	6–6		9	14
`Add.d F9 F1 F1`	5	6–7		10	14
`Mult.d F10 F1 F1`	6	7–10		16	17
`Ld F11 8(R0)`	6	7–7	8–9	15	17
`Addi R6 R2 20`	7	8–8		13	18
`Add.d F12 F2 F2`	7	8–9		14	18
`Mult.d F13 F2 F1`	8	9–12		18	19
`Ld F14 12(R0)`	8	9–9	10–11	17	19

Figure 10: Simulation results for Test Case #03 with issue width 2 (19 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Add R3 R1 R2`	1	2–2		3	4
`Add.d F3 F1 F2`	1	2–3		4	5
`Mult.d F4 F1 F2`	1	2–5		8	9
`Ld F5 0(R0)`	2	3–3	4–5	9	10
`Sub R4 R2 R1`	2	3–3		5	10
`Sub.d F6 F2 F1`	2	3–4		6	10
`Mult.d F7 F2 F2`	3	4–7		13	14
`Ld F8 4(R0)`	3	4–4	6–7	14	15
`Addi R5 R1 10`	3	4–4		7	15
`Add.d F9 F1 F1`	4	5–6		10	15
`Mult.d F10 F1 F1`	4	5–8		15	16
`Ld F11 8(R0)`	4	5–5	8–9	17	18
`Addi R6 R2 20`	5	6–6		11	18
`Add.d F12 F2 F2`	5	6–7		12	18
`Mult.d F13 F2 F1`	5	6–9		16	19
`Ld F14 12(R0)`	6	7–7	10–11	18	19

Figure 11: Simulation results for Test Case #03 with issue width 3 (19 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Add R3 R1 R2`	1	2–2		3	4
`Add.d F3 F1 F2`	1	2–3		4	5
`Mult.d F4 F1 F2`	1	2–5		10	11
`Ld F5 0(R0)`	1	2–2	3–4	8	11
`Sub R4 R2 R1`	2	3–3		5	11
`Sub.d F6 F2 F1`	2	3–4		6	11
`Mult.d F7 F2 F2`	2	3–6		13	14
`Ld F8 4(R0)`	2	3–3	5–6	14	15
`Addi R5 R1 10`	3	4–4		7	15
`Add.d F9 F1 F1`	3	4–5		9	15
`Mult.d F10 F1 F1`	3	4–7		15	16
`Ld F11 8(R0)`	3	4–4	7–8	17	18
`Addi R6 R2 20`	4	5–5		11	18
`Add.d F12 F2 F2`	4	5–6		12	18
`Mult.d F13 F2 F1`	4	5–8		16	18
`Ld F14 12(R0)`	4	5–5	9–10	18	19

Figure 12: Simulation results for Test Case #03 with issue width 4 (19 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–7		8	9
`Sd F3 0(R1)`	2	3–3	11–13		11
`Ld F4 0(R1)`	3	4–4	9	10	12
`Addi R2 R0 4`	4	5–5		6	13
`Addi R2 R2 4`	5	7–7		9	14
`Addi R2 R2 4`	6	10–10		11	15
`Addi R2 R2 4`	8	12–12		13	16
`Sd F1 0(R2)`	9	14–14	18–20		18
`Ld F5 16(R0)`	10	11–11	15	16	19
`Add.d F6 F5 F4`	11	17–18		19	20

Figure 13: Simulation results for Test Case #04 with issue width 1 (20 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–7		8	9
`Sd F3 0(R1)`	1	2–2	11–13		11
`Ld F4 0(R1)`	2	3–3	9	10	11
`Addi R2 R0 4`	2	3–3		4	12
`Addi R2 R2 4`	3	5–5		6	12
`Addi R2 R2 4`	4	7–7		9	13
`Addi R2 R2 4`	6	10–10		11	13
`Sd F1 0(R2)`	6	12–12	15–17		15
`Ld F5 16(R0)`	7	8–8	13	14	15
`Add.d F6 F5 F4`	7	15–16		17	18

Figure 14: Simulation results for Test Case #04 with issue width 2 (18 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–7		8	9
`Sd F3 0(R1)`	1	2–2	11–13		11
`Ld F4 0(R1)`	1	3–3	9	10	11
`Addi R2 R0 4`	2	3–3		4	11
`Addi R2 R2 4`	2	5–5		6	12
`Addi R2 R2 4`	4	7–7		9	12
`Addi R2 R2 4`	6	10–10		11	12
`Sd F1 0(R2)`	6	12–12	14–16		14
`Ld F5 16(R0)`	6	7–7	13	14	15
`Add.d F6 F5 F4`	7	15–16		17	18

Figure 15: Simulation results for Test Case #04 with issue width 3 (18 cycles).

	ISSUE	EX	MEM	WB	COMMIT
`Mult.d F3 F1 F2`	1	2–7		8	9
`Sd F3 0(R1)`	1	2–2	11–13		11
`Ld F4 0(R1)`	1	3–3	9	10	11
`Addi R2 R0 4`	1	2–2		3	11
`Addi R2 R2 4`	2	4–4		5	11
`Addi R2 R2 4`	3	6–6		7	12
`Addi R2 R2 4`	5	8–8		9	12
`Sd F1 0(R2)`	5	10–10	14–16		14
`Ld F5 16(R0)`	5	6–6	11	12	14
`Add.d F6 F5 F4`	5	13–14		15	16

Figure 16: Simulation results for Test Case #04 with issue width 4 (16 cycles).

Final values	Integer registers (non-zero)	R1 = 8, R2 = 16
	FP registers (non-zero)	F1 = 2, F2 = 5, F3 = 10, F4 = 10, F5 = 2, F6 = 12
	Memory (non-zero)	Mem[8] = 10, Mem[16] = 2

Both loads have a MEM stage of one cycle at every width, which means they took their value from a store. `F5` ends up as 2 (the value of `F1` stored by the second store) and not the old memory value 7, which matches the sequential model.

6 Conclusion

I extended the starter simulator to issue up to 4 instructions per cycle, while keeping its structure. I also added the handout's input format and the register and memory output, and I added the 1-bit branch predictor with recovery from the handout, and fixed the starter bugs that broke results with several instructions in flight. Wider issue helped most when independent instructions were available, but the single CDB, branch mispredictions and long dependence chains limited the gain. Across the tests, I got the same final values and committed instruction sequence as the sequential model at every issue width I checked.